

3

Converting a Xamarin.Forms App into .NET MAUI

Before we dive too far into .NET MAUI, we will look back at an existing Xamarin.Forms app and convert it into .NET MAUI. This chapter will guide you through the steps to convert an existing Xamarin.Forms app running on Mono into a .NET MAUI app running on .NET 7. We will discuss two different methods for converting your Xamarin.Forms application into .NET MAUI. The first method will use a new .NET MAUI project and move our old Xamarin.Forms code into the new project. The second method will use the **.NET Upgrade Assistant** tool to do some of the upgrades for us.

If you are new to .NET MAUI and are not coming from Xamarin.Forms app development, feel free to skip this chapter and go straight to the next project.

The following topics will be covered in this chapter:

- Moving code to a new .NET MAUI project
- An overview of upgrading a Xamarin.Forms app
- Installing and running .NET Upgrade Assistant

Technical requirements

To complete this project, you need to have Visual Studio installed on your **Macintosh (Mac)** or PC, as well as the .NET Mobile components. Refer to *Chapter 1, Introduction to .NET MAUI*, for more details on how to set up your environment. There are additional components that will be installed in this chapter, so you will need an internet connection to download and install .NET Upgrade Assistant. This chapter provides screenshots and instructions for Visual Studio on Windows.

This chapter will both be a classic **File | New | Project** chapter and use an existing app, guiding you step by step through the process of migrating an app from Xamarin.Forms to .NET MAUI. For the second app, you will need to download the source from this book's GitHub repository.

You can find the full source for the code in this chapter at <https://github.com/PacktPublishing/MAUI-Projects-3rd-Edition> under the **Chapter03** folder.

Project overview

This chapter is not meant to be an exhaustive tome of all the things you need to be aware of when converting your Xamarin.Forms app into .NET MAUI. Rather, it is an overview of what you need to consider when migrating your app, along with two walk-throughs of methods to accomplish that task. There are too many variations in application styles, versions, frameworks, custom controls, and so on for this one chapter to cover every scenario. That could take an entire book and would most likely be outdated by the time it was published. So, in this chapter, we are going to focus on a simple migration method that has the benefit of using the .NET MAUI Single Project feature and .NET Migration Assistant, which will automate much of the manual stuff and is constantly being updated.

The first part of this chapter will use a new Xamarin.Forms app created from the Shell template. The second part of this chapter will use .NET Upgrade Assistant to upgrade the open source app, BuildChat, which is available on GitHub at <https://github.com/mindofai/Build2019Chat>.

Every development book needs to have a chat app; this book is no different. For .NET Migration Assistant, we will use an existing chat app that was built using Xamarin.Forms. The app can be debugged and tested locally, so there is no need to set up and configure any cloud services.

The build time for this project is about one hour.

Migrating into a blank .NET MAUI template

This method of moving your existing app code into a new .NET MAUI app is used mainly for smaller, simpler apps that do not have a lot of external dependencies, such as NuGet, or native libraries. The largest benefit of this method is that your migrated app will be in a single project, targeting all the .NET MAUI-supported platforms. If your original app only targeted Android and iOS, using this method could get you Mac Catalyst and Windows targets for free. Using .NET Upgrade Assistant will not add any platforms that you didn't already target.

To illustrate these steps, we will create a new project, just like we did in *Chapter 2*. This time, however, we will have to create a Xamarin.Forms project first.

Creating a new Xamarin.Forms app

The following steps will guide you through creating a new Xamarin.Forms project:

1. Open Visual Studio 2022 and select **Create a new project**:

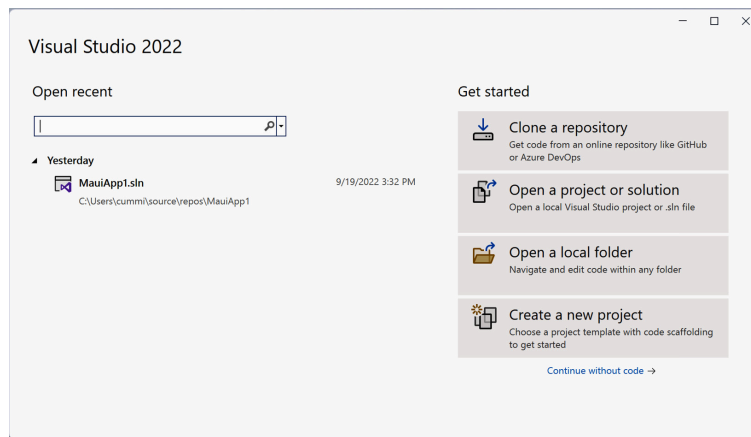


Figure 3.1 – Visual Studio 2022

This will open the **Create a new project** wizard.

2. In the search field, type **Xamarin.Forms** and select the **Mobile App (Xamarin.Forms)** item from the list:

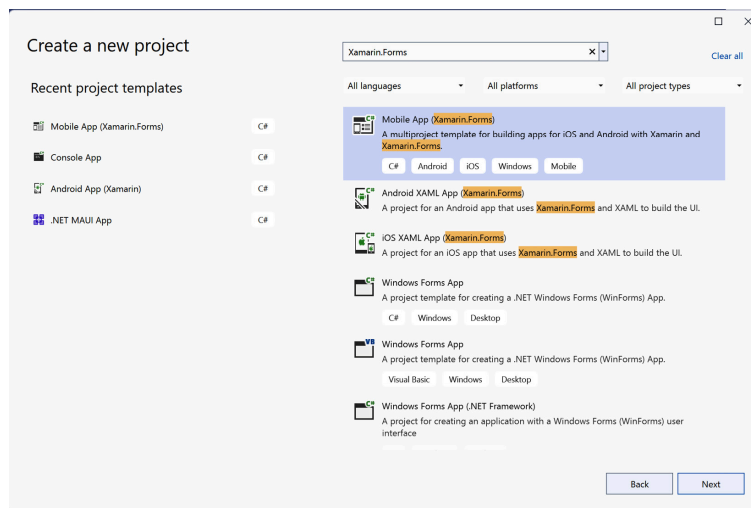


Figure 3.2 – New Xamarin.Forms project

3. Complete the next page of the wizard by naming your project **MauiMigration**, and then click **Next**:

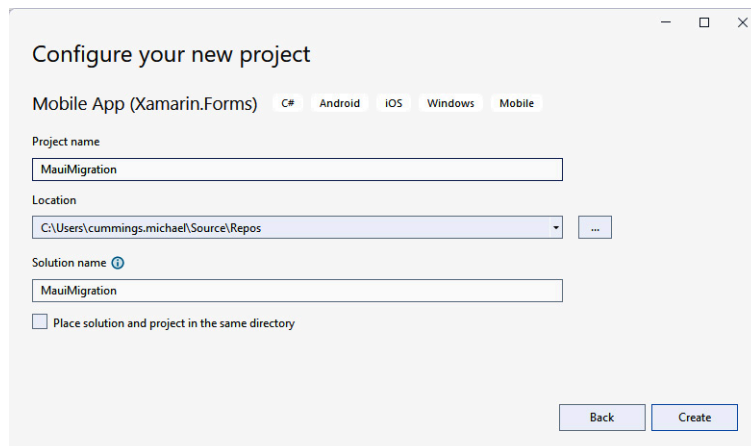


Figure 3.3 – Configuring the Xamarin.Forms project

4. Select the **Flyout** template and ensure that all three Xamarin.Forms platforms are checked:

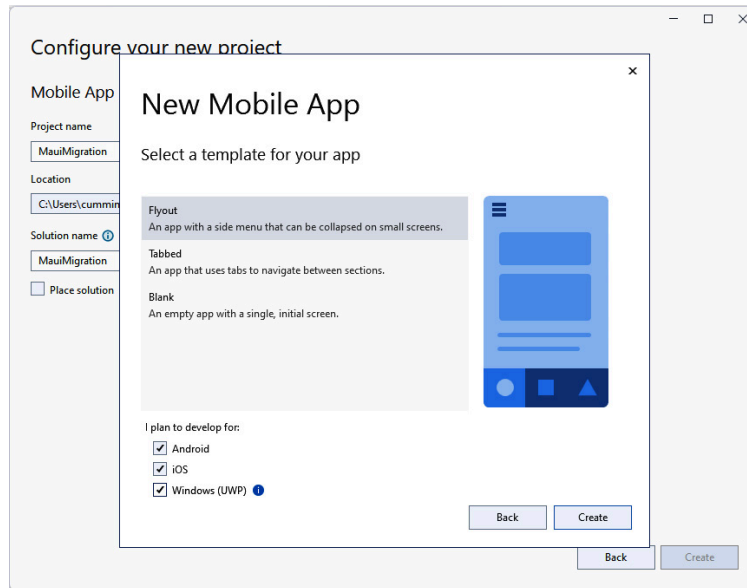


Figure 3.4 – Selecting the Xamarin.Forms template and platforms

You may receive a few messages about out-of-support components. These are expected if this is the first time you are creating a Xamarin.Forms app and can be safely ignored.

5. Finalize the setup by clicking **Create** and wait for Visual Studio to create the project.
Before we begin to migrate this app, it's a good idea to make sure it works properly.
6. Run the app and test out all the buttons, flyout options, and menus.
On the **About** page, there is a **Learn more** button that will open a browser and navigate you to the Xamarin.Forms quickstart web page:

Learn more at <https://aka.ms/xamarin-quickstart>

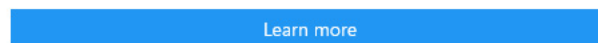


Figure 3.5 – The Learn more button

7. The flyout menu has three options: **About**, **Browse**, and **Logout**.
Ensure you click on each one and explore all their functionality:

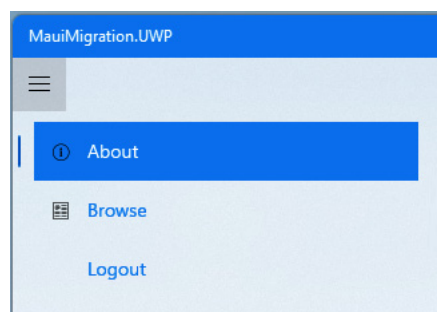


Figure 3.6 – Flyout menu options

Now that we have explored this Xamarin.Forms app, let's move on to creating the new .NET MAUI project, which will act as our new app.

Creating a new .NET MAUI app

We will add a new project to the current solution to make things easier. To create a new .NET MAUI project, follow these steps:

1. In Visual Studio, right-click the **Mauimigration** solution item in **Solution Explorer** and select **File | Add | New Project**:

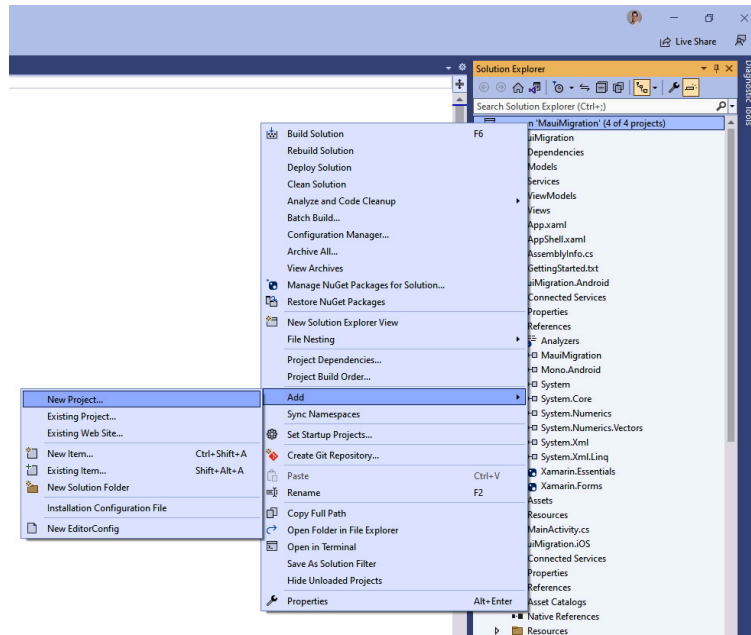


Figure 3.7 – Adding a new project to solution

2. In the **Add a new project** dialog, select **.NET MAUI App** and click **Next**:

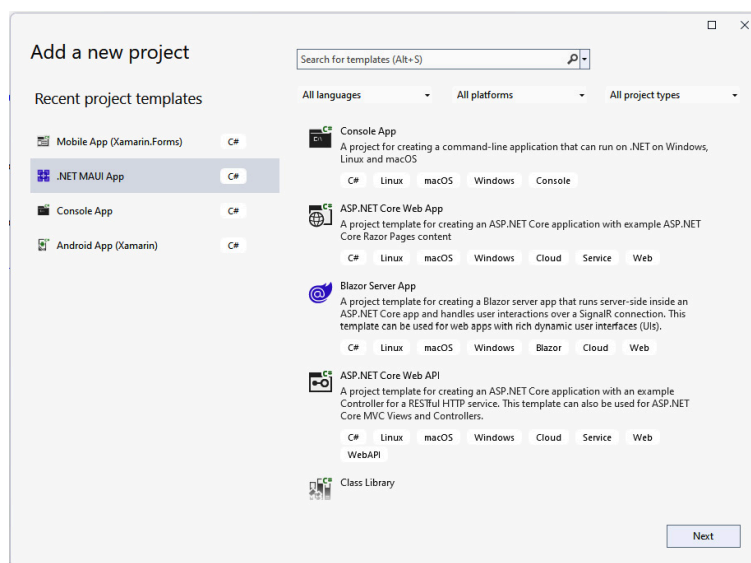


Figure 3.8 – Adding a new .NET MAUI project

3. In the **Configure your new project** dialog, set the project name to **MyMauApp** and click **Next**:

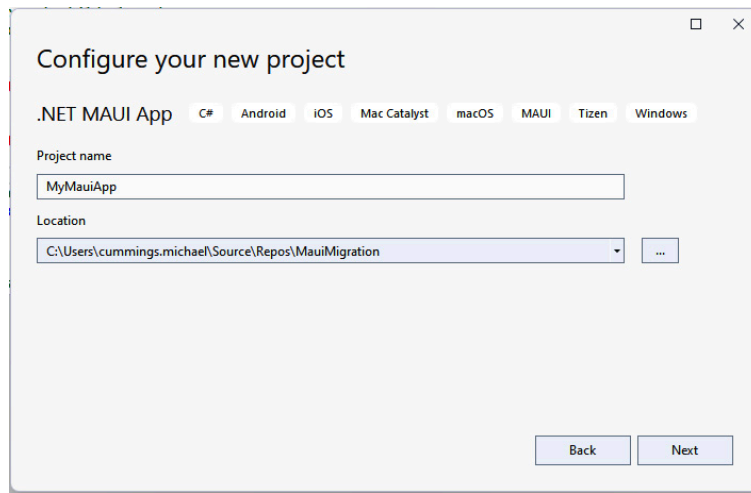


Figure 3.9 – Configuring the .NET MAUI project

4. In the **Additional Information** dialog, ensure that the selected framework is **.NET 7.0 (Standard Term Support)** and click **Create**.

Now that we have the shell of the .NET MAUI app, we can start moving the important parts of the Xamarin.Forms app over to the .NET MAUI project.

Migrating the MauiMigration app to MyMauiApp

Migrating a Xamarin.Forms app into a blank new .NET MAUI template will involve the following high-level steps:

1. Copy your app files into the new template.
2. Change the Xamarin.Forms namespace to its .NET MAUI equivalent.
3. Update the app startup so that it uses your views.

The next few sections will explain how to accomplish these steps.

Copying files to a new project

To begin, let's copy the XAML and C# files and folders from the Xamarin.Forms project to the .NET MAUI project. In the **MauiMigration** project, select the **Models**, **Services**, **ViewModels**, and **Views** folders. We will copy these files instead of moving them so that we don't destroy the original project. Right-click any of the selected folders and select **Copy**. Right-click the **MauiApp** project and click **Paste**.

A few images are also needed; we will copy them from the **MauiMigration.UWP** project. Under the **MauiMigration.UWP** project, you will find three image files called **icon_about.png**, **icon_feed.png**, and **xamarin_logo.png**. Select all three files and copy them just like you did for the previous files. Paste the files into the **MyMauiApp/Resources/Images** folder.

Visual Studio may make some changes to your project file when you copy/paste files, such as adding new item groups that remove and add the same file to the project. You can safely remove these changes as the single

project system knows how to handle **XAML** and **.png** files. If you get compilation errors related to missing images or errors in the **XAML** files, check **MyMauiApp.csproj** for any additional **ItemGroups** that reference either the **.png** files or **XAML** files and remove them.

The following screenshot shows an example of the **MyMauiApp.csproj** file after copying the images from **MauiMigration.UWP**. *Figure 3.10* shows the changes that Visual Studio added; these can be removed:

```

46      <!-- Raw Assets (also remove the "Resources\Raw" prefix) -->
47      <MauiAsset Include="Resources\Raw\*" LogicalName="%%(RecursiveDir)%(Filename)%(Extension)" />
48      </ItemGroup>
49
50      <ItemGroup>
51      <MauiImage Remove="Resources\Images\icon_about.png" />
52      <MauiImage Remove="Resources\Images\icon_feed.png" />
53      <MauiImage Remove="Resources\Images\xamarin_logo.png" />
54      </ItemGroup>
55
56      <ItemGroup>
57      <None Remove="Resources\Images\icon_about.png" />
58      <None Remove="Resources\Images\icon_feed.png" />
59      <None Remove="Resources\Images\xamarin_logo.png" />
60      </ItemGroup>
61
62      <ItemGroup>
63      <AndroidResource Include="Resources\Images\icon_about.png" />
64      <AndroidResource Include="Resources\Images\icon_feed.png" />
65      <AndroidResource Include="Resources\Images\xamarin_logo.png" />
66      </ItemGroup>
67
68      <ItemGroup>
69      <PackageReference Include="Microsoft.Extensions.Logging.Debug" Version="7.0.0" />
70      </ItemGroup>
71
72      </Project>
73
74

```

Figure 3.10 – Visual Studio added unnecessary items

Next, we will need to update the **XAML** files so that they reference .NET MAUI controls.

Updating namespaces

Currently, the **XAML** files are still using the Xamarin.Forms namespace. To update these files, we need to change the following:

```
xmlns="http://xamarin.com/schemas/2014/forms"
```

We must amend this like so:

```
xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
```

You can use the **Find and Replace** feature in Visual Studio to make these changes or manually edit each file. To use the **Find and Replace** dialog, follow these steps:

1. In **Solution Explorer**, select the **MyMauiApp** project.
2. From the Visual Studio menu select **Edit | Find and Replace | Replace in Files** (or **Ctrl + Shift + H**) to open the **Find and Replace** dialog:

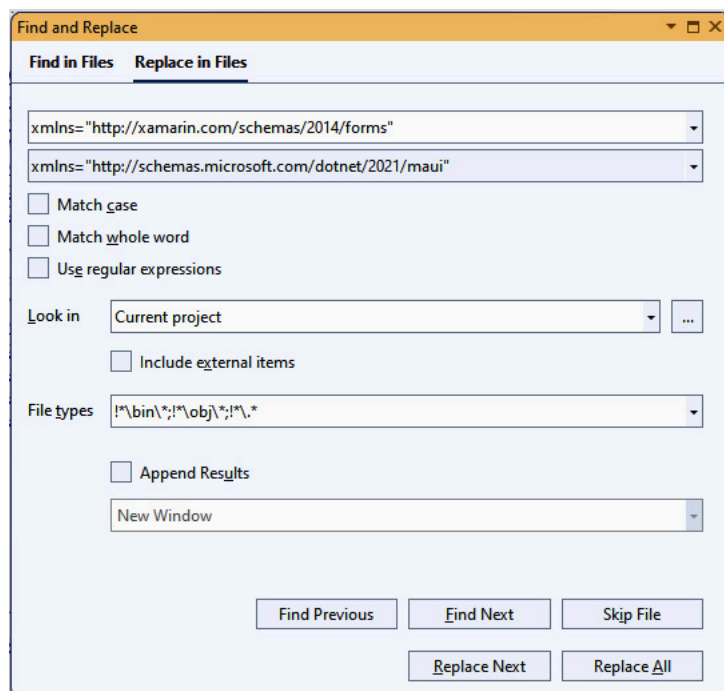


Figure 3.11 – The Find and Replace dialog

3. In the **Look in** field, select **Current project**.
4. Click the **Replace All** button; there should only be five places to make that change.

Once completed, there are a few other changes that need to be made.

In the `NewItemPage.xaml` file, in the `Views` folder of the `MyMauiApp` project, remove the following highlighted text:

```
Title="New Item"
xmlns:ios="clr-namespace:Xamarin.Forms.PlatformConfiguration.iOSSpecific;assembly=Xamarin.Forms.Co
ios:Page.UseSafeArea="true">
```

Now that the **XAML** file namespace changes are complete, we can move on to the **C#** namespace changes.

Using the **Find and Replace** dialog again, we can remove all the `Xamarin.Forms` namespace references. This time, by using a regular expression, we can remove multiple lines. Use the following expression in the **Find** entry box:

```
^using Xamarin\.[Forms,Essentials].*;
```

Then, check the **Use regular expressions** checkbox and select **Current project** for **Look in**, as shown here:

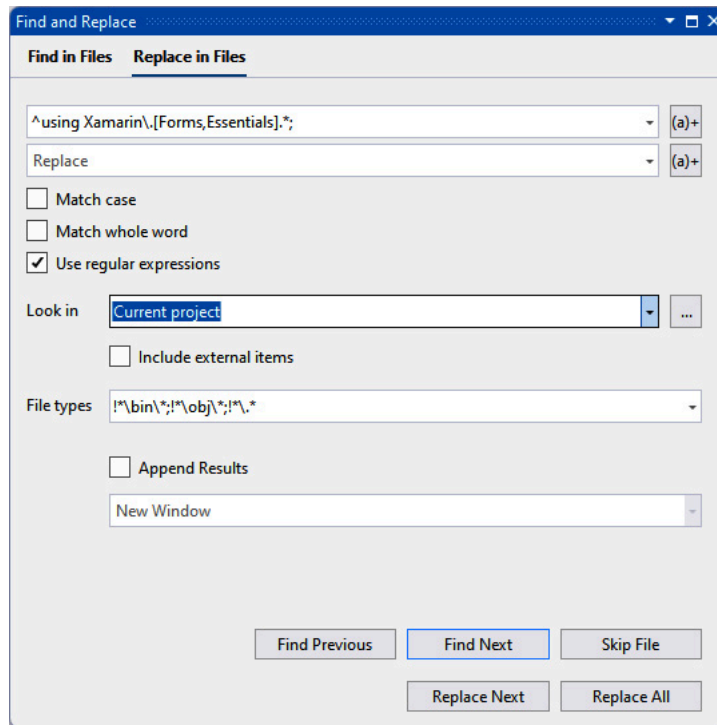


Figure 3.12 – Find and Replace – C# namespaces

Most of the changes have been made, and the app should compile at this point. However, it is not using any of the code we have copied and modified as .NET MAUI Shell does not reference the copied pages. The next step is to hook Shell into our code.

Modifying the app startup

The **MyMauiApp** project is still using the default **MainPage.xaml** file as its startup page. The next step in our migration is to make the **AppShell1.xaml** file the same as it is for our Xamarin.Forms app.

What we need to copy is in the **MauiMigration** project, in the **AppShell1.xaml** file, starting at line 76 until line 101, as shown in the following code snippet:

```
<!--
    When the Flyout is visible this defines the content to display in the flyout.
    FlyoutDisplayOptions="AsMultipleItems" will create a separate flyout item for each child e
-->
<FlyoutItem Title="About" Icon="icon_about.png">
    <ShellContent Route="AboutPage" ContentTemplate="{DataTemplate local:AboutPage}" />
</FlyoutItem>
<FlyoutItem Title="Browse" Icon="icon_feed.png">
    <ShellContent Route="ItemsPage" ContentTemplate="{DataTemplate local:ItemsPage}" />
</FlyoutItem>
<!-- When the Flyout is visible this will be a menu item you can tie a click behavior to -->
<MenuItem Text="Logout" StyleClass="MenuItemLayoutStyle" Clicked="OnMenuItemClicked">
</MenuItem>
<!--
    TabBar lets you define content that won't show up in a flyout menu. When this content is a
    the flyout menu won't be available. This is useful for creating areas of the application w
    you don't want users to be able to navigate away from. If you would like to navigate to th
```

```

        content you can do so by calling
        await Shell.Current.GoToAsync("//LoginPage");
    -->
    <TabBar>
        <ShellContent Route="LoginPage" ContentTemplate="{DataTemplate local:LoginPage}" />
    </TabBar>

```

Copy the preceding code and replace the following lines in the **MyMauiApp** project's **AppShell.xaml** file:

```

    <ShellContent
        Title="Home"
        ContentTemplate="{DataTemplate local:MainPage}"
        Route="MainPage" />

```

To enable the flyout, you will need to remove the following highlighted text in **AppShell.xaml**:

```

xmlns:local="clr-namespace:MyApp"
Shell.FlyoutBehavior="Disabled">

```

The final change that we need to make will register the correct instance for the **Browse** page to use for the list of items. Open the **MauiProgram.cs** file and make the changes highlighted in the following code block:

```

using Microsoft.Extensions.Logging;
using MauiMigration.Services;
using MauiMigration.Models;
namespace MyMauiApp
{
    public static class MauiProgram
    {
        public static MauiApp CreateMauiApp()
        {
            var builder = MauiApp.CreateBuilder();
            builder
                .UseMauiApp<App>()
                .ConfigureFonts(fonts =>
                {
                    fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
                    fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
                });
            DependencyService.RegisterSingleton<IDataStore<Item>>(new MockDataStore());
#if DEBUG
            builder.Logging.AddDebug();
#endif
            return builder.Build();
        }
    }
}

```

At this point, you should be able to compile and run the converted project. Make sure you set **MyMauiApp** as the startup project before running. Play around with the app and make sure it is all working before moving on to the next section on manually migrating your apps to .NET MAUI.

Manual migration overview

In the previous section, we converted a simple Xamarin.Forms app into .NET MAUI using the Single Project system. This project did not use any advanced features of Xamarin.Forms, such as external NuGets, custom controls, or any commercial controls. These are additional items that you will have to consider when migrating your apps from Xamarin.Forms to .NET MAUI.

In this section, we are going to discuss the basic flow that you should go through to migrate your Xamarin.Forms app to .NET MAUI. This is by no means an all-inclusive list; the .NET MAUI team is updating a wiki page that details all their knowledge in one location.

OFFICIAL MIGRATION GUIDE

The guidelines for migrating your Xamarin.Forms apps to .NET MAUI are continually evolving since the release of .NET MAUI, based on usage and feedback. To review the latest guidelines, please visit the following URL in your favorite browser: <https://learn.microsoft.com/en-us/dotnet/maui/get-started/migrate?view=net-maui-7.0>.

When migrating your apps from Xamarin.Forms to .NET MAUI, you will need to follow the overall steps outlined here:

1. Convert the Xamarin.Forms projects from .NET Framework into .NET SDK style.
2. Update the code from Xamarin.Forms to .NET MAUI.
3. Update any incompatible dependencies with .NET 6+ versions.
4. Address any breaking API changes.
5. Run the converted app and verify its functionality.

.NET Upgrade Assistant is a tool that will attempt to perform the first four steps for you. However, before we dive into using .NET Upgrade Assistant, we are going to look at what each of these steps entails so that we have a firm understanding of how to migrate our app if .NET Upgrade Assistant is unable to operate on an app project.

Converting the Xamarin.Forms projects from .NET Framework into .NET SDK style

Some Xamarin.Forms projects are based on the .NET Framework project template. It is a verbose project format that has been updated for .NET projects. The new format, typically referred to as the SDK style, is a much more concise format with better defaults. Xamarin.Forms projects created with Visual Studio 16.5 or later use the newer SDK format.

To convert an old format project file, such as the one we will be using with .NET Upgrade Assistant, in the *Installing and running .NET Upgrade Assistant* section later in this chapter, you would need to change the `<Project />` element to the new SDK style, like this:

1. Find a line like this in your project file:

```
<Project ToolsVersion="4.0" DefaultTargets="Build" xmlns="http://schemas.microsoft.com/develope
```

2. Replace it with the following:

```
<Project Sdk="Microsoft.NET.Sdk">
```

Remember that you will have to make this change for all your Xamarin.Forms projects, platform-specific projects, and any shared library projects.

Let's look at the specific changes that you might have to make for a typical Xamarin.Forms project that targets both Android and iOS.

To convert the shared library project of a Xamarin.Forms app into a .NET MAUI project, we need to replace the contents of the **csproj** file with the following:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFrameworks>net7.0-ios;net7.0-android; </TargetFrameworks>
    <TargetFrameworks Condition="$([MSBuild]::IsOSPlatform('windows'))">$(TargetFrameworks);net7.0-windows10.0.19041.0</TargetFrameworks>
    <UseMaui>True</UseMaui>
    <OutputType>Library</OutputType>
    <ImplicitUsings>enable</ImplicitUsings>
    <!-- Required for C# Hot Reload -->
    <UseInterpreter Condition="'$(Configuration)' == 'Debug'">True</UseInterpreter>
    <SupportedOSPlatformVersion Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) == 'ios'">14.2</SupportedOSPlatformVersion>
    <SupportedOSPlatformVersion Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) == 'android'">21.0</SupportedOSPlatformVersion>
    <SupportedOSPlatformVersion Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) == 'windows'">10.0.17763.0</SupportedOSPlatformVersion>
    <TargetPlatformMinVersion Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) == 'windows'">10.0.17763.0</TargetPlatformMinVersion>
  </PropertyGroup>
</Project>
```

The important bits are highlighted. **<UseMaui>True</UseMaui>** will enable the project system to automatically add the correct references to the project for .NET MAUI libraries. The **<TargetFrameworks>** elements are updated for the correct **Target Framework Monikers (TFMs)** for .NET 6 or 7.

.NET MAUI also requires an additional **<SupportedOSPlatformVersion>** property, which is set conditionally based on **TargetPlatformIdentifier**.

That is a significant amount of reduction from the .NET Framework **csproj** files. In addition to the property reduction, you can remove most of the **<ItemGroup>...</ItemGroup>** entries since all source files are now

included by default. The groups to leave are the groups containing `<ProjectReferences />` entries.

Now that we've converted the shared project, let's review the changes needed for an Android project.

As with all .NET Framework projects, we will need to change the `<Project ...>` element to the following:

```
<Project Sdk="Microsoft.NET.Sdk">
```

Now, remove all the `<PropertyGroup>...</PropertyGroup>` elements since they are default values, and replace them with the following:

```
<PropertyGroup>
  <UseMaui>True</UseMaui>
  <TargetFramework>net7.0-android</TargetFramework>
  <OutputType>Exe</OutputType>
  <ImplicitUsings>enable</ImplicitUsings>
  <SupportedOSPlatformVersion Condition="'$(TargetFramework)' == 'net7.0-android'">31.0</SupportedOSPlatformVersion>
</PropertyGroup>
<PropertyGroup>
  <UseInterpreter Condition="$(TargetFramework.Contains('-android'))">True</UseInterpreter>
</PropertyGroup>
```

Again, in the Android project, we set the `<UseMaui>` and `<TargetFramework>` properties.

To finish migrating this project file, remove all the `<ItemGroup>` elements and their contents, except for the one group that contains the `<AndroidResource>` elements. Finally, remove the `<Import>` element at the bottom of the project file; it is no longer needed.

The next project type to look at is an iOS project. The changes here will be very similar to the changes in an Android project but with an iOS twist. Xamarin.Forms iOS projects are also a .NET Framework style project, so we will need to change the `<Project ...>` element to the following:

```
<Project Sdk="Microsoft.NET.Sdk">
```

Then, we will need to remove all the existing `<PropertyGroup>` elements and add the following:

```
<PropertyGroup>
  <UseMaui>true</UseMaui>
  <TargetFramework>net7.0-ios</TargetFramework>
  <OutputType>Exe</OutputType>
  <ImplicitUsings>enable</ImplicitUsings>
  <SupportedOSPlatformVersion
Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(
TargetFramework)')) == 'ios'">14.2</SupportedOSPlatformVersion>
</PropertyGroup>
```

Again, you will notice the `<UseMaui>` property being set, and that the `<TargetFramework>` value is `net7.0-ios`.

To finish converting an iOS project, remove the `<ItemGroup>` and `<Import>` elements except for the one containing the `<ProjectReference>` items; those are still needed.

Updating code from Xamarin.Forms to .NET MAUI

Updating the code from Xamarin.Forms to .NET MAUI consists of several steps. First, we need to add some new code that is required to initialize .NET MAUI. Refer to the *Examining the files* section of *Chapter 2* for more details on the files that are needed.

The first change you will need is in the shared project. Add a new class file called **MauiProgram.cs** with the following contents, minimally:

```
public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf",
                    "OpenSansRegular");
            });
        return builder.Build();
    }
}
```

You may have to add additional code to this file so that it can handle additional needs for your application, such as registering types for dependency injection or registering additional library features, such as logging.

In the **Platforms/Android** folder, add a new class file named **MainApplication.cs** and update the class so that it looks like this:

```
[Application]
public class MainApplication : MauiApplication
{
    public MainApplication(IntPtr handle, JniHandleOwnership ownership)
        : base(handle, ownership)
    {
    }
    protected override MauiApp CreateMauiApp() => MauiProgram.CreateMauiApp();
}
```

We must also make a few adjustments to the **MainActivity** class. Open the **MainActivity.cs** file and make the changes highlighted in the following code:

```
[Activity(Label = "ManualMigration", Icon = "@mipmap/icon",
Theme = "@style/Theme.MaterialComponents", MainLauncher = true,
ConfigurationChanges = ConfigChanges.ScreenSize | ConfigChanges.
Orientation | ConfigChanges.UiMode | ConfigChanges.ScreenLayout |
ConfigChanges.SmallestScreenSize )]
public class MainActivity : MauiAppCompatActivity
{
    protected override void OnCreate(Bundle savedInstanceState)
    {
        base.OnCreate(savedInstanceState);
    }
}
```

The final change we would need to make in the Android project is to update **targetSdkVersion** in the **AndroidManifest.xml** file to version **33**, like this:

```
<uses-sdk android:minSdkVersion="21" android:targetSdkVersion="33" />
```

ANDROID SDK VERSIONS

*Since the Android SDK is updated yearly, it may be the case that the version of .NET for Android and .NET MAUI that you are using is also using a version of the Android SDK that is greater than 33. The good news is that you will get an error in Visual Studio if **targetSdkVersion** is too low or is not installed. Just follow the instructions in the build error to set the SDK version correctly.*

That is all the changes we need to make in the Android project for now. Moving on to the iOS project, the **AppDelegate.cs** file can be updated so that it matches the following:

```
public partial class AppDelegate : MauiUIApplicationDelegate
{
    protected override MauiApp CreateMauiApp() => MauiProgram.
CreateMauiApp();
}
```

The final change for the iOS project is to open the **Info.plist** file and change the **MinimumOSVersion** property to **15.2**.

With the base changes needed to start your app as a .NET MAUI app done, the next changes are much broader brush strokes:

1. Remove all **Xamarin.*** namespaces from **.cs** files.
2. Change all **xaml** namespace declarations from the following:

```
xmlns="http://xamarin.com/schemas/2014/forms"
```

You will need to amend them like so:

```
xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
```

You may notice that these are the same changes we made in the previous section when using a .NET MAUI Single Project.

ABOUT IMAGES

.NET MAUI has improved image handling for the various platforms that it targets. You can provide a single SVG image file, and it will resize the image correctly for all platforms for you. Since the SVG format is based on vectors, it will render much better than other formats such as JPG and PNG after re-sizing. It is recommended that you convert your images into SVG format, if possible, to take advantage of this feature in .NET MAUI.

Updating any incompatible NuGet packages

There are a lot of NuGet packages out there and there is no way we can cover them all. But, in general, for each of the NuGet packages that are in use in your app, be sure to look for a version that specifically supports .NET MAUI or the version of .NET that you are targeting. You can use the NuGet Gallery page to determine whether a package supports .NET MAUI. Using a popular package such as PCLCrypto version 2.0.147 (<https://www.nuget.org/packages/PCLCrypto/2.0.147#supportedframeworks-body-tab>) targets classic Xamarin projects but not .NET 6 or .NET 7. You can find the compatible frameworks under the **Frameworks** tab:

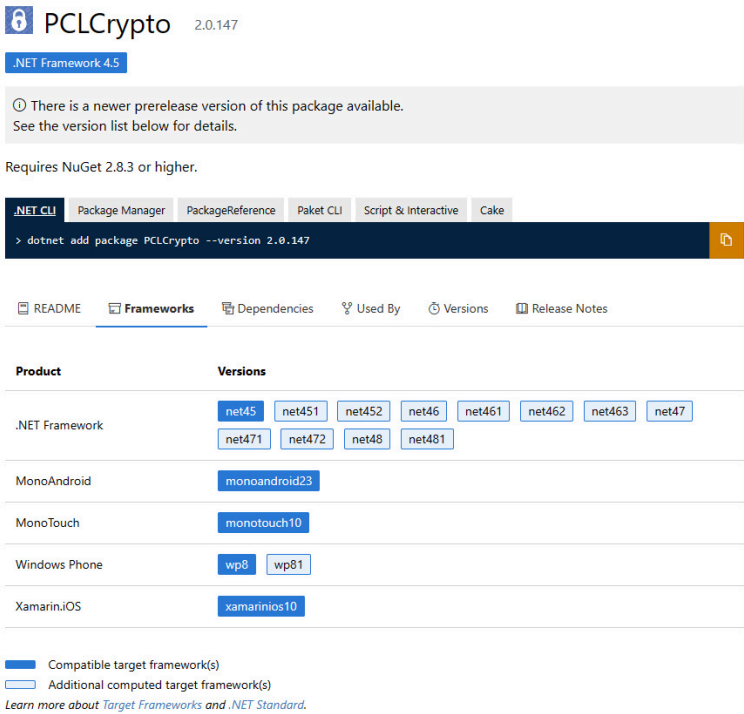


Figure 3.13 – NuGet Gallery page for PCLCrypto v2.0.147

However, version 2.1.40-alpha (<https://www.nuget.org/packages/PCLCrypto/2.1.40-alpha#supportedframeworks-body-tab>) lists .NET 6 and .NET 7 as compatible frameworks:

PCLCrypto 2.1.40-alpha

.NET Core 2.1 .NET Standard 2.0 .NET Framework 4.7.2

ⓘ This is a prerelease version of PCLCrypto.

Package Manager PackageReference Paket CLI Script & Interactive Cake

```
> dotnet add package PCLCrypto --version 2.1.40-alpha
```

README Frameworks Dependencies Used By Versions Release Notes

Product	Versions
.NET	net5.0 net5.0-windows net6.0 net6.0-android net6.0-ios net6.0-maccatalyst net6.0-macos net6.0-tvos net6.0-windows net7.0 net7.0-android net7.0-ios net7.0-maccatalyst net7.0-macos net7.0-tvos net7.0-windows
.NET Core	netcoreapp2.0 netcoreapp2.1 netcoreapp2.2 netcoreapp3.0 netcoreapp3.1
.NET Standard	netstandard2.0 netstandard2.1
.NET Framework	net461 net462 net463 net47 net471 net472 net48 net481
MonoAndroid	monoandroid monoandroid90
MonoMac	monomac
MonoTouch	monotouch
Tizen	tizen40 tizen60
Xamarin.iOS	xamarinios xamarinios10
Xamarin.Mac	xamarinmac
Xamarin.TVOS	xamarintvos
Xamarin.WatchOS	xamarinwatchos

Compatible target framework(s)
Additional computed target framework(s)
[Learn more about Target Frameworks and .NET Standard.](#)

Figure 3.14 – NuGet Gallery page for PCLCrypto v2.1.40-alpha

Currently, we know of the following NuGet changes:

- Remove all Xamarin.Forms and Xamarin.Essentials NuGet references from your projects. These are now included in .NET MAUI directly. You will have to make some namespace adjustments as those have changed.
- Replace Xamarin.Community Toolkit with the latest preview of .NET MAUI Community Toolkit. You will have to make some namespace adjustments as those have changed.
- If you reference any of the following SkiaSharp NuGet packages directly, replace them with the latest previews:
 - **SkiaSharp.Views.Maui.Controls**
 - **SkiaSharp.Views.Maui.Core**
 - **SkiaSharp.Views.Maui.Controls.Compatibility**

You can find the latest version of NuGet packages on the NuGet Gallery website at <https://nuget.org/packages>.

Addressing any breaking API changes

Unfortunately, there is no magic bullet for any of these types of changes. You will simply have to start from the top of your error list and work your way through them. You can review the release notes linked in the official migration guide, available at <https://learn.microsoft.com/en-us/dotnet/maui/migration/>, as helpful hints.

For example, a common type of error that's seen is **error CS0104:**

'ViewExtensions' is an ambiguous reference between 'Microsoft.Maui.Controls.ViewExtensions' and 'Microsoft.Maui.ViewExtensions'. This can be fixed by explicitly using the full namespace when referencing the type or by using a type alias – for example, **using ViewExtensions = Microsoft.Maui.Controls.ViewExtensions.**

Custom renderers and effects

Your application may use custom renderers or effects to provide a unique user experience. Covering how to upgrade these components is beyond the scope of this chapter. To learn more about how to upgrade renderers and effects, visit the Microsoft Learn site for .NET MAUI migration at

<https://learn.microsoft.com/en-us/dotnet/maui/migration/>.

Running the converted app and verifying its functionality

This is not the last step – I recommend that you attempt to do this after each change. Building your app as you make changes ensures that you are moving in the right direction. I recommend that you **rebuild** over **building** and even go so far as to delete the **obj** and **bin** folders beforehand. This will ensure that you are building with the latest changes and dependencies, by forcing a NuGet restore.

Now that we know the basics of how to convert a Xamarin.Forms app, let's use .NET Upgrade Assistant to migrate a project for us.

Installing and running .NET Upgrade Assistant

As stated previously, .NET Upgrade Assistant will attempt to perform the first four steps of the migration of your Xamarin.Forms app to .NET MAUI outlined in the previous section. The tool is under active development and as the team discovers new improvements, they are added. This is mostly due to feedback they receive from developers like you.

At the time of writing, .NET Upgrade Assistant did not work on all projects and has the following limitations:

- Xamarin.Forms must be version 5.0 and higher
- Only Android and iOS projects are converted
- .NET MAUI must be properly installed with the appropriate workloads

If you have followed the steps from *Chapter 1*, then the last should already be satisfied.

If your Xamarin.Forms app meets these criteria, then we can get started by installing the tool.

Installing .NET Upgrade Assistant

.NET Upgrade Assistant is a Visual Studio extension on Windows and a command-line tool on Windows and macOS. You can use the integrated developer PowerShell in Visual Studio or any command-line prompt to install the tool. Follow these steps to install the tool using Visual Studio on Windows:

1. In Visual Studio, select the **Extensions** menu, then the **Manage Extensions** item. This will open the **Manage Extensions** dialog.
2. In the **Manage Extensions** dialog, use the search box to search for **up-grade**.
3. Select **.NET Upgrade Assistant** and click **Download**:

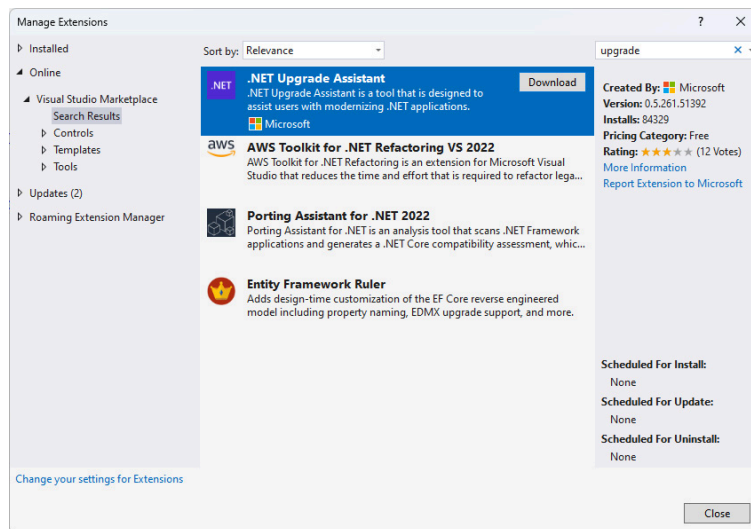


Figure 3.15 – Visual Studio – the Manage Extensions dialog

4. Once the extension has been downloaded, you will need to close and reopen Visual Studio to install the extension:

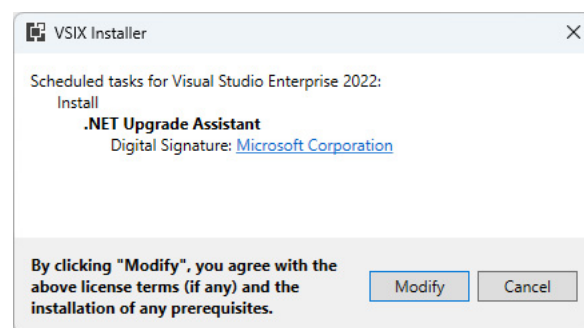


Figure 3.16 – Installing the .NET Upgrade Assistant VSIX

5. Click **Modify** and then follow the instructions to complete the installation.
6. Once the installation is complete, reopen Visual Studio.

Now that the tool has been installed, we can use it to convert a Xamarin.Forms project!

Preparing to run .NET Upgrade Assistant

For the remainder of this chapter, we will be using Visual Studio on Windows to migrate a Xamarin.Forms app to .NET MAUI.

To run .NET Upgrade Assistant, we will need a Xamarin.Forms project to upgrade. For this portion of the chapter, we will use the Xamarin.Forms app that was demoed at the Microsoft Build conference in 2019. The source can be found at <https://github.com/mindofai/Build2019Chat>, though you can find it in this book's GitHub repository at <https://github.com/PacktPublishing/MAUI-Projects-3rd-Edition> under the `Chapter03/Build2019Chat` folder.

Once you have downloaded the source, open the `BuildChat.sln` file in Visual Studio. Once the project has finished loading, make sure your configuration is correct by running the app first. You should see a screen that looks like this on Android:

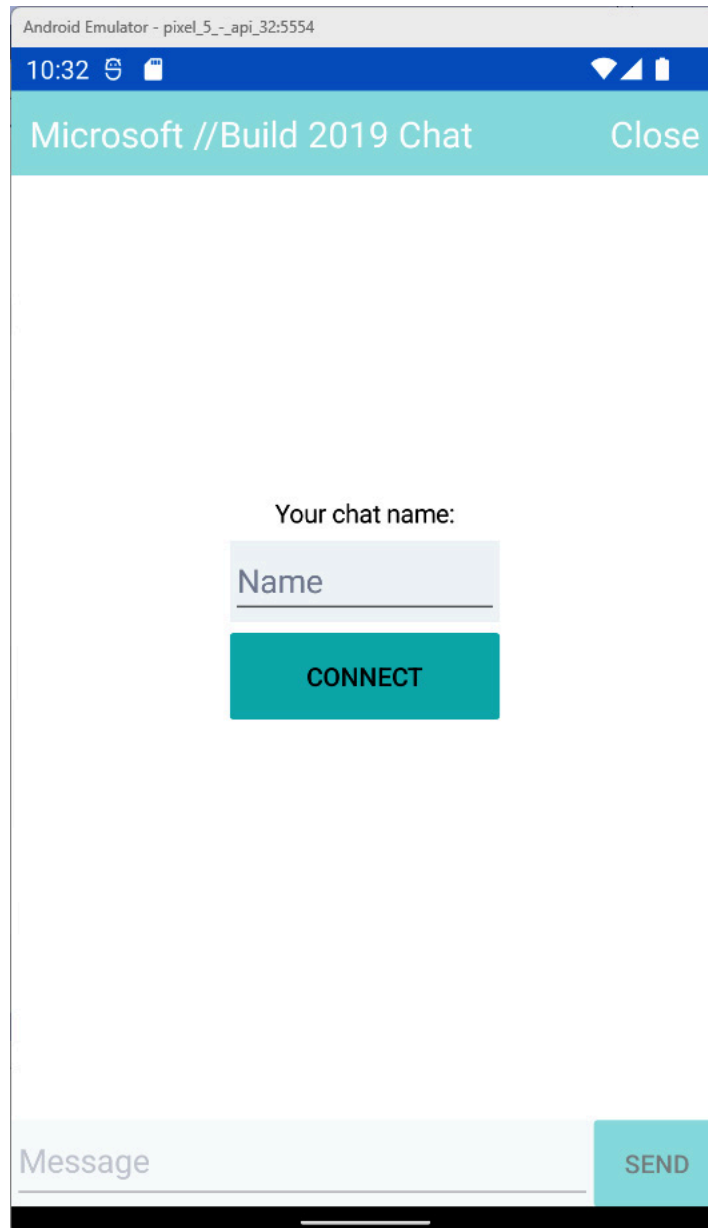


Figure 3.17 – The original app on Android

Now that we have confirmed that the original app runs, we can follow these steps to prepare for running .NET Upgrade Assistant:

1. Right-click the **BuildChat** solution node in **Solution Explorer** and select **Manage NuGet Packages for Solution...**:

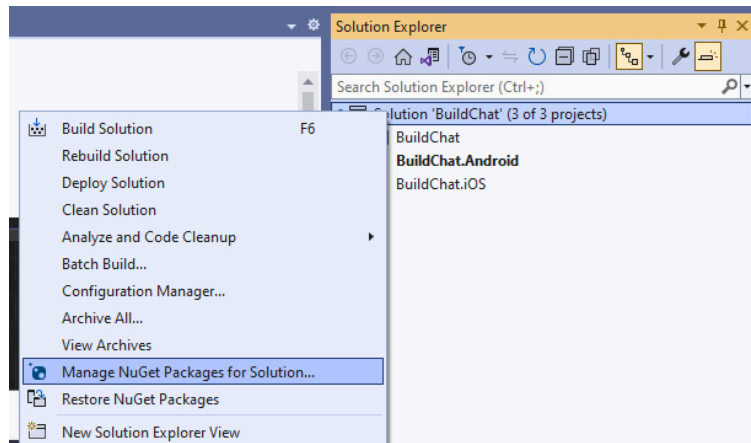


Figure 3.18 – Solution context menu

2. In the **NuGet – Solution** window that opens, select the **Updates** tab:

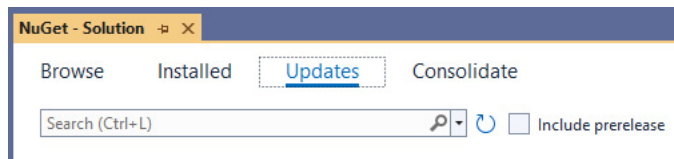


Figure 3.19 – The NuGet – Solution window

3. Click the **Select all packages** checkbox, then click **Update**.
4. Visual Studio will prompt you with a preview of all the changes that will be made. Click **OK** once you have reviewed them.
5. Visual Studio will then prompt you to accept the license terms for packages that have them. Once you have reviewed the license terms, click **I Accept**.
6. After updating, you may still have a **gold bar** indicator in the Visual Studio window from running the application earlier. You can safely dismiss the message by clicking the **X** button on the right:

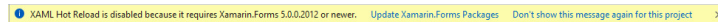


Figure 3.20 – Xamarin.Forms version gold bar

7. Once the packages have been updated, let's make sure the app is still working by running it again. You should get a build error like the following:

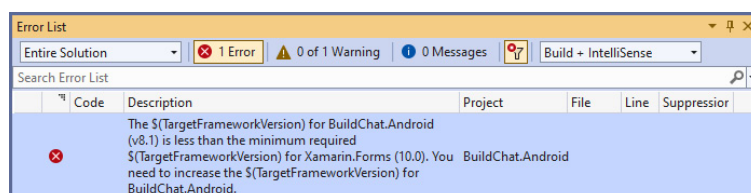


Figure 3.21 – Error after upgrading packages

8. To resolve this error, in **Solution Explorer**, select the **BuildChat.Android** project, then press *Alt* and *Enter* at the same time to open the project properties page.
9. Use the **Target Framework** dropdown to change the value from **Android 8.1 (Oreo)** to **Android 10.0**.

GOOGLE PLAY SUPPORT

*You may get a warning about Google Play requiring new apps and updates to support a specific version of Android. To remove that warning, just set **Target Framework** to the version indicated in the warning message.*

10. Visual Studio will prompt you to confirm the change as it has to close and re-open the project. Select **Yes**.
11. Visual Studio may also prompt you to install the Android version if you haven't installed it. Follow the prompts to install the Android version.
12. Attempting to run the project again yields a new set of errors:

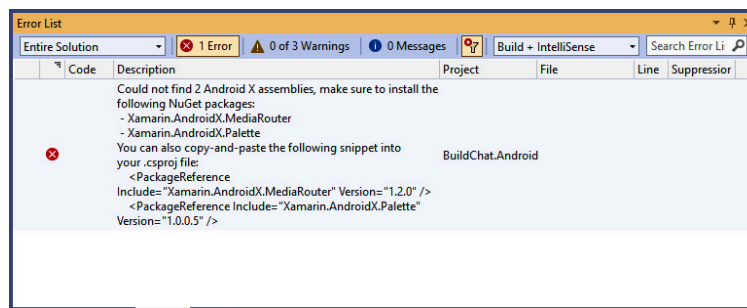


Figure 3.22 – Missing packages error

13. To resolve this error, right-click the **BuildChat.Android** project and select **Unload Project**. The project file should open in Visual Studio automatically.
14. Locate **<ItemGroup>** in the file with **<PackageReference>** items and make the changes highlighted in the following snippet:

```
<ItemGroup>
  <PackageReference Include="Microsoft.AspNetCore.SignalR.Client">
    <Version>7.0.9</Version>
  </PackageReference>
  <PackageReference Include="Xamarin.Forms" Version="5.0.0.2578" />
  <PackageReference Include="Xamarin.Android.Support.Design" Version="28.0.0.3" />
  <PackageReference Include="Xamarin.Android.Support.v7.AppCompat" Version="28.0.0.3" />
  <PackageReference Include="Xamarin.Android.Support.v4" Version="28.0.0.3" />
  <PackageReference Include="Xamarin.Android.Support.v7.CardView" Version="28.0.0.3" />
  <PackageReference Include="Xamarin.AndroidX.MediaRouter" Version="1.2.0" />
  <PackageReference Include="Xamarin.AndroidX.Palette" Version="1.0.0.5" />
</ItemGroup>
```

You could also use Visual Studio's NuGet Package Manager to add these packages.

15. Save and reload the project before trying to run it again. Since the project was unloaded, you will need to set the **BuildChat.Android** project as the startup project again.

You should be able to run the application at this time since some warnings can be ignored. If not, review the previous steps to make sure you made all the changes correctly. At this point, we are ready to run the upgrade assistant to convert from Xamarin.Forms into .NET MAUI.

TREAT WARNINGS AS ERRORS

If you have the project option to treat warnings as errors set to anything other than none, then the warnings will prevent you from running the app. Set the option to none to allow the app to run. The option defaults to none.

Running .NET Upgrade Assistant

Running .NET Upgrade Assistant from within Visual Studio is a straightforward process. We will upgrade each project individually; there isn't any method to upgrade all the projects in one go.

Upgrading the BuildChat project

Let's start with the shared project, **BuildChat**, by following these steps:

1. Select the **BuildChat** project in **Solution Explorer**.
2. Use the context menu to select the **Upgrade** menu item:

Figure 3.23 – Upgrading the BuildChat project

This will open the **Upgrade** assistant in a document window:

Figure 3.24 – Upgrading the BuildChat project

3. Select the **In-place project upgrade** option.
4. Depending on the versions of .NET you have installed, you will be prompted to choose one. If you followed the setup instructions in *Chapter 1*, you should have the .NET 7.0 option available. Select **.NET 7.0** and select **Next**:

Figure 3.25 – Choosing the preferred target framework

5. At this point, you are allowed to review the changes that will be made by expanding each node in the list. You can also choose to not upgrade certain items by removing the check in the checkbox next to that item. When you have inspected all the changes, make sure all items are checked again, then click **Upgrade selection**:

Figure 3.26 – Reviewing the upgrade

6. Visual Studio will start the upgrade process. You can monitor it as it completes each item:

Figure 3.27 – Upgrade in progress

7. When it's finished, you can inspect each item to see what the result of the upgrade was:

Figure 3.28 – Upgrade complete

A white check in a green circle indicates some transformation was completed and successful, a green check with a white background means the step was skipped since nothing was needed, and a red cross (not shown) means the transformation failed. You can view the complete output from the tool by inspecting the **Upgrade Assistant** log in the output pane:

Figure 3.29 – Upgrade Assistant log output

Do not be concerned with the errors in the error window at this point. There will be errors until we finish upgrading the remaining projects. Now that the **BuildChat** project has been upgraded, we can upgrade the **BuildChat.Android** project.

Upgrading the BuildChat.Android project

The steps for upgrading the remaining projects are largely the same – the only difference will be the steps involved in upgrading each project. The next two sections will skip the screenshots and just provide the steps. To complete the upgrade for the **BuildChat.Android** project, follow these steps:

1. Select the **BuildChat.Android** project in **Solution Explorer**.
2. Use the context menu to select the **Upgrade** menu item.
3. This will open the **Upgrade** assistant in a document window.
4. Select the **In-place project upgrade** option.
5. Select the **.NET 7.0** option, then select **Next**.
6. Review the changes that will be made by expanding each node in the list. Make sure all items are checked, then click **Upgrade Selection**.
7. Visual Studio will complete the upgrade process.

Now that .NET Upgrade Assistant has completed the **BuildChat.Android** project, we can upgrade the **BuildChat.iOS** project.

Upgrading the BuildChat.iOS project

The steps for upgrading the iOS project are largely the same – the only difference will be the steps involved in upgrading each project. To complete the upgrade for the **BuildChat.iOS** project, follow these steps:

1. Select the **BuildChat.iOS** project in **Solution Explorer**.
2. Use the context menu to select the **Upgrade** menu item.
3. This will open the **Upgrade** assistant in a document window:
4. Select the **In-place project upgrade** option.
5. Select the **.NET 7.0** option, then select **Next**.
6. Review the changes that will be made by expanding each node in the list. Make sure all items are checked, then click **Upgrade Selection**.
7. Visual Studio will complete the upgrade process.

Now that .NET Upgrade Assistant has completed the **BuildChat.iOS** project, we can see how well it worked.

Completing the upgrade to .NET MAUI

With .NET Upgrade Assistant having done all the work it can to upgrade the projects, we can now see what is left for us to complete the upgrade to .NET MAUI.

The first thing we want to do is make sure that the project is clean of all the previous build artifacts. This will ensure we are referencing all the right dependencies in our build output by forcing a restore and build. The best way to accomplish this is to remove the **bin** and **obj** folders from each project folder.

Use **File Explorer** to remove the **bin** and **obj** folders from the **BuildChat**, **BuildChat.Android** and **BuildChat.iOS** folders, then build the solution.

We'll end up with a few build errors for each project, as shown in the following figure:

Figure 3.30 – Package issues

To resolve these errors, either use Visual Studio's NuGet Package Manager to add a reference to version 7.0.1 of the **Microsoft.Extensions.Logging.Abstractions** package to all the projects, or follow these steps to update the project files manually:

1. Select the **BuildChat** project in **Solution Explorer**.

The project file will open in a document window automatically.

2. Locate the **ItemGroup** element that contains the **PackageReference** items.
3. Make the changes highlighted in the following snippet:

```
<ItemGroup>
  <PackageReference Include="Microsoft.AspNetCore.SignalR.Client" Version="7.0.9" />
  <PackageReference Include="Microsoft.Extensions.Logging.Abstractions" Version="7.0.1" />
</ItemGroup>
```

4. Select the **BuildChat.Android** project in **Solution Explorer**.

The project file will open in a document window automatically.

5. Locate the **ItemGroup** element that contains the **PackageReference** items.
6. Make the changes highlighted in the following snippet:

```
<ItemGroup>
  <PackageReference Include="Xamarin.AndroidX.MediaRouter" Version="1.2.0" />
  <PackageReference Include="Xamarin.AndroidX.Palette" Version="1.0.0.5" />
  <PackageReference Include="Microsoft.AspNetCore.SignalR.Client" Version="7.0.9" />
  <PackageReference Include="Microsoft.Extensions.Logging.Abstractions" Version="7.0.1" />
</ItemGroup>
```

7. Select the **BuildChat.iOS** project in **Solution Explorer**.

The project file will open in a document window automatically.

8. Locate the **ItemGroup** element that contains the **PackageReference** items.

9. Make the changes highlighted in the following snippet:

```
<ItemGroup>
  <PackageReference Include="Microsoft.AspNetCore.SignalR.Client" Version="7.0.9" />
  <PackageReference Include="Microsoft.Extensions.Logging.Abstractions" Version="7.0.1" />
</ItemGroup>
```

Now that we have added the required package references, we can try building the app again. After this build, we'll get two new errors:

Figure 3.31 – Namespace does not exist errors

These two errors show two areas that the upgrade assistant did not upgrade. Luckily, we covered how to upgrade these two files easier in this chapter. Let's upgrade them again, starting with the **BuildChat.Android** project.

Open the **MainActivity.cs** file and make the changes highlighted in the following code:

```
using Microsoft.Maui;
namespace BuildChat.Droid
{
    [Activity(Label = "BuildChat", Icon = "@mipmap/icon", Theme = "@style/Theme.MaterialComponents
    public class MainActivity : MauiAppCompatActivity
    {
        protected override void onCreate(Bundle savedInstanceState)
        {
            base.onCreate(savedInstanceState);
        }
    }
}
```

That should complete the changes needed for the Android project. Now, to upgrade the iOS project, open the **AppDelegate.cs** file in **BuildChat.iOS** and update it so that it matches the following:

```
using Foundation;
using Microsoft.Maui;
using Microsoft.Maui.Hosting;
namespace BuildChat.iOS
{
    // The UIApplicationDelegate for the application. This class is responsible for launching the
    // User Interface of the application, as well as listening (and optionally responding) to
    // application events from iOS.
    [Register("AppDelegate")]
    public partial class AppDelegate : MauiUIApplicationDelegate
    {
        protected override MauiApp CreateMauiApp() => MauiProgram.CreateMauiApp();
    }
}
```

```
}  
}
```

The final change for the iOS project is to open the **Info.plist** file and change the **MinimumOSVersion** property to **15.2**. To make this change, use **Generic PList Editor**. To open the **Info.plist** file and make this change, follow these steps:

1. Select the **Info.plist** file in the **BuildChat.iOS** project.
2. Use the context menu (right-click) and select **Open With....**
3. In the **Open With** dialog, select **Generic Plist Editor**, then select **OK**:

Figure 3.32 – Opening the Info.plist file

4. Find the entry labeled **Minimum system version** and change the value from **8.0** to **15.1**:

Figure 3.33 – Changing the minimum system version

Great – that should complete the changes needed to get the app running as a .NET MAUI application! The following are the before and after screenshots of the application; *before* is on the left and *after* is on the right:

Figure 3.34 – Xamarin.Forms versus .NET MAUI

There are some visual changes between Xamarin.Forms and .NET MAUI, and you can tweak the .NET MAUI settings for the layouts and controls to get a very similar output.

Summary

In this chapter, we focused on upgrading a Xamarin.Forms app to .NET MAUI. We learned how to upgrade the project files from .NET Framework to SDK-style projects, application startup files, XAML views, and C# files needed for .NET MAUI. We started by doing this manually to learn about all the required steps and changes. We ended this chapter by using .NET Upgrade Assistant to make many of the changes for us. We also learned how to upgrade to the Single Project format, which is the default for .NET MAUI. Now, we can pick the best method for the project we are upgrading.

While we covered a lot in this chapter, it was not exhaustive. There is a lot of variation in different projects, from NuGet package dependencies and vendor-provided controls to customizations using renderers and effects. The app you are upgrading may use one or all of these, and you can find additional help on the Microsoft Learn site for upgrading Xamarin.Forms to .NET MAUI: <https://learn.microsoft.com/en-us/dotnet/maui/migration/>.

If you are interested in seeing the **BuildChat** app fully functional, try using .NET Upgrade Assistant on the service that was also built for the 2019 Microsoft Build conference. You can find the source on GitHub at <https://github.com/mindofai/SignalRChat/tree/master>. You could also use ChatGPT to help you build the service yourself using Azure Functions and SignalR.

In the next chapter, we will build an app that displays news articles using the new .NET MAUI Shell.